

3. 프롬프팅 기초 개념 및 구조적 설계

1. 프롬프팅의 기초 개념

프롬프트의 요소와 구조

How a Prompt Travels through an LLM Service

2. Zero-shot 프롬프트

2.1. 개념 설명

Zero-shot 프롬프트란 예시(example)를 전혀 포함하지 않고 모델에게 직접적인 지시만으로 작업을 요청하는 프롬프트를 의미합니다. 말 그대로 "한 번에(zero-shot)" 모델에게 바로 질문이나 명령을 던지는 방식입니다. 이런 경우 프롬프트에는 문제 설명이나 요청만 있고 문제에 대한 예시나 데모는 포함되지 않습니다. 모델은 오로지 사전에 훈련된 지식과 언어 이해능력에 의존하여 답을 생성하게 됩니다.

대규모로 학습된 최신 LLM들은 명령어만으로도 꽤 많은 작업을 수행할 수 있는 zero-shot 능력을 보여줍니다. 다만 프롬프트가 막연하거나 모델이 해당 작업에 익숙하지 않은 경우, 출력이 부정확하거나 일관성이 떨어질 수 있습니다. Zero-shot 프롬프트는 설정이 간단하고 토큰을 적게 사용하지만, 모델이 바로 이해하지 못하는 복잡한 과제에는 한계가 있을 수 있습니다. 그런 경우 다음에 다룰 few-shot 기법으로 프롬프트에 예시를 추가하면 성능 향상을 도모할 수 있습니다.

2.2. 프롬프트 예시

아래는 Zero-shot 프롬프트의 실무형 예시입니다. 예를 들어, 고객 리뷰 문장을 분류하는 간단한 감성 분석 작업을 zero-shot으로 요청할 수 있습니다. 모델에게 별도의 예시를 주지 않고 바로 문장의 감정을 분류하도록 지시합니다:

예시 프롬프트:

다음 문장의 감정을 긍정, 부정 또는 중립 중 하나로 분류하세요.

문장: "이 제품은 쓸만하지만 가격이 너무 비싸요."

감정:

위와 같이 문장과 분류할 감정 레이블만 제시하면, 모델은 자체 학습한 지식을 활용하여 해당 문장의 감정을 판단합니다. 이 경우 예상되는 예시 응답은 다음과 같습니다.

예시 응답: 부정

아무런 예시 없이도 모델은 "가격이 너무 비싸다"라는 내용에서 부정적 감정임을 추론해 "부정"이라고 답할 것입니다. 이처럼 Zero-shot 프롬프트는 추가 힌트나 사례 없이 바로 작업을 요청하므로, 프롬프트 문장 자체를 명확하게 작성하는 것이 중요합니다.

2.3. Playground 실습 가이드

##실습 3.1. OpenAI Playground Zero-shot 실습

Zero-shot 프롬프트를 Playground에서 직접 실습해보겠습니다. 다음 안내에 따라 단계별로 실행하세요:

1. **OpenAI Playground 접속:** 브라우저에서 OpenAI Playground 페이지를 열고 로그인합니다 (* 사전에 OpenAI API Key가 설정되어 있어야 함).
2. **모델 선택:** 상단 모델 선택 드롭다운에서 **GPT-4o-mini** 모델을 선택합니다 (기본 모델로 설정되어 있을 수 있습니다).
3. **프롬프트 입력:** 입력 창에 위 예시 프롬프트 내용을 그대로 입력합니다. 예시처럼 한국어로 입력해도 되고, 모델은 한국어도 이해하므로 그대로 활용해도 좋습니다.
4. **실행:** 우측의 **Submit** 또는 **Generate** 버튼을 눌러 프롬프트를 실행합니다. 모델이 응답을 생성할 때까지 잠시 기다립니다.
5. **결과 확인:** 모델이 출력한 응답을 확인합니다. 정상적으로 작동했다면, 한 단어로 **부정**과 같이 감정 레이블을 답변했을 것입니다.
6. **비교 실습:** 동일한 프롬프트를 이번엔 **GPT-4o** 모델로 실행해보세요 (모델 메뉴에서 GPT-4o 선택). 두 모델의 응답 속도와 결과를 비교하면서, 보다 성능이 높은 모델이 **zero-shot** 지시를 더 정확히 수행하는지 관찰합니다.

노트: Zero-shot 프롬프트의 성능은 프롬프트 표현의 명확성에 크게 의존합니다. 실습 시 프롬프트 문구를 조금 바꿔보거나 다른 간단한 과제를 물어보면서, 모델이 어떻게 반응하는지 실험해보세요. 예를 들어, "간단한 날씨 설명을 해줘." 같은 질문도 zero-shot 프롬프트에 해당하며, 이때 모델이 어떻게 답하는지 관찰합니다.

##실습 3.2. OpenAI Platform 모델별 Pricing 확인 실습

3. Few-shot 프롬프트

3.1. 개념 설명

Few-shot 프롬프트는 프롬프트에 모델에게 보여줄 여러 개의 예시(demonstrations)를 포함하여 모델이 **맥락 속에서 학습(in-context learning)**할 수 있도록 하는 기법입니다. 즉, 두 개 이상의 예시를 질문 전에 제공함으로써 모델이 **패턴을 인식**하고 더 나은 응답을 생성하도록 유도합니다. 예시에는 질문-답변 쌍이나 원하는 출력 형식이 담긴 샘플 등이 포함되며, 모델은 이들을 참고하여 사용자 요청에 대한 해결책을 추론합니다.

Few-shot 프롬프트의 장점은 모델이 주어진 예시들을 통해 작업의 맥락과 형식을 파악하여 **zero-shot보다 응답의 일관성을 높일 수** 있다는 점입니다. 특히 복잡하거나 모호한 작업의 경우 몇 가지 예시만으로도 모델의 출력 품질이 크게 향상될 수 있습니다. 이는 모델이 추가 훈련없이 **프롬프트 내 예시만으로 학습**할 수 있는 능력(ICL)을 활용하는 것입니다.

예를 들어, 요구하는 출력 형식이 엄격하거나 특별한 패턴이 필요한 경우 few-shot으로 여러 사례를 보여주면 모델이 그 패턴을 따를 가능성이 높아집니다. 다만 **주의점**으로는, 예시를 많이 넣을수록 프롬프트 토큰 길이가 늘어나기 때문에 비용이 증가할 수 있습니다. 따라서 **필요한 만큼의 최소한의 예시**를 포함하는 것이 좋으며, 예시들은 **요구 작업과 밀접하게 연관**되고 **다양한 경우를 대표**할 수 있도록 선택하는 것이 효과적입니다.

3.2. 프롬프트 예시

Few-shot 프롬프트의 작성을 이해하기 위해, 앞서 Zero-shot에서 사용한 **감정 분류 작업**에 예시를 추가해 보겠습니다. 이번에는 모델에게 두 가지 **샘플 문장과 그 감정 분류 결과**를 먼저 보여준 후, 새로운 문장의 감정을 묻는 형식으로 프롬프트를 구성합니다:

예시 프롬프트:

다음 문장의 감정을 긍정, 부정 또는 중립 중 하나로 분류하세요.

문장: "이 제품은 정말 좋아요."

감정: 긍정

문장: "별로 기대에 못 미칩니다."

감정: 부정

문장: "이 제품은 쓸만하지만 가격이 너무 비싸요."

감정:

위 프롬프트에는 두 개의 **예시 입력과 그에 대한 모델의 이상적인 출력**이 포함되어 있습니다. 모델은 첫 번째 문장 "정말 좋아요."가 **긍정**, 두 번째 "기대에 못 미칩니다."가 **부정**으로 분류된 것을 본 뒤, 세 번째 새로운 문장에 대해 답해야 합니다. 이러한 예시를 통해 모델은 **어떤 단어와 표현이 긍정 또는 부정을 나타내는지** 힌트를 얻게 되고, **출력 형식** (감정 레이블만 한 단어로 답함)도 학습하게 됩니다.

이 프롬프트에 대한 **예시 응답**으로 모델은 다시 "부정"이라고 답할 것이 기대됩니다. Few-shot 구성 덕분에 모델은 세 번째 문장을 앞서 본 예시들과 비교하여 일관된 방식으로 분류하게 됩니다. 이처럼 Few-shot 프롬프트는 **모델에게 작업의 예시를 직접 보여주어** 더 정확하고 원하는 형태의 결과를 얻는 강력한 방법입니다.

3.3. Playground 실습 가이드

Few-shot 프롬프트를 Playground에서 실습해봅시다. Zero-shot 때보다 프롬프트 작성 이 길어지므로, 단계별로 주의 깊게 따라하세요:

##실습 3.3. OpenAI Playground 모델별 실습

1. **모델 선택 확인:** OpenAI Playground에서 **GPT-4o-mini** 모델이 선택되어 있는지 확인합니다 (변경하지 않았다면 이전 설정이 유지되어 있을 것입니다).
2. **프롬프트 작성:** 입력 창에 위 **예시 프롬프트**의 내용을 입력합니다. 여러 줄의 예시를 포함하므로, 각 줄이 정확히 구분되도록 입력하세요. (복사/붙여넣기 할 경우 줄바꿈이 제대로 되었는지 확인합니다.)
3. **프롬프트 검토:** 프롬프트에 포함된 **예시 문장과 레이블**, 그리고 마지막에 분류할 새 문장이 올바르게 작성됐는지 다시 한번 확인합니다. 예시가 논리적으로 연결되고 형식이 동일해야 모델이 혼란 없이 학습할 수 있습니다.
4. **생성 실행:** **Submit** 버튼을 눌러 모델에 프롬프트를 보냅니다. Zero-shot보다 프롬프트 길이가 길지만 GPT-4o-mini 모델은 충분히 처리가 가능합니다.
5. **응답 확인:** 모델의 출력이 **마지막 문장에 대한 감정 레이블 하나로** 나오는지 확인합니다. 제대로 작동했다면 **부정** 과 같은 단어 하나를 답변으로 받을 것입니다. Few-shot 예시를 준 덕분에 모델이 Zero-shot 때와 동일하게 정확히 대답했다면 성공입니다.
6. **추가 실험:** Few-shot 프롬프트의 효과를 느껴보기 위해 예시를 변형해보세요. 예를 들어, **예시의 감정 표현을 영어(positive/negative)로 바꾸고** 모델이 영어로 출력하도록 하거나, 예시 문장을 1개만(one-shot) 혹은 3개로 늘려보는 실험을 합니다. 예시의 수나 내용이 바뀔 때 모델 응답의 변화가 있는지 관찰해 보세요.
7. **고급 모델 비교:** 가능하다면 **GPT-4o** 모델로도 동일한 프롬프트를 실행해 보세요. **더 강력한 모델일수록 few-shot에서 예시의 미묘한 차이도 잘 학습하는지**, 또는 GPT-4o-mini와 결과가 어떻게 다른지 비교해보는 것도 도움이 됩니다.

Tip: Few-shot 프롬프트에 포함하는 예시는 반드시 정확하고 일관된 답을 담고 있어야 합니다. 모델은 예시로 주어진 내용을 정답으로 신뢰하기 때문에, 잘못된 예시를 주면 그 패턴을 따라갈 수 있습니다. 또한 예시는 다양할수록 좋지만, 너무 길거나 많으면 오히려 모델이 혼란을 느낄 수 있으니 균형을 맞추는 것이 중요합니다.

4. 역할 지시 프롬프트 (Role Prompting)

4.1. 개념 설명

역할 지시 프롬프트(Role Prompting)는 모델에게 특정한 역할이나 인격을 부여하여, 그 관점과 말투로 답변을 생성하도록 유도하는 기법입니다. 쉽게 말해 모델에게 "~로서 행동하라"는 지시를 내리는 것입니다. 예를 들어, "당신은 이제부터 역사 교사입니다..."라고 프롬프트하면 모델은 역사 교사의 말투와 지식을 활용해 답변하려고 합니다. 이렇게 **LLM에게 페르소나(인격)**를 설정해주면 응답의 **스타일, 톤, 전문성 수준**을 그 역할에 맞게 조정할 수 있습니다. 일반적으로 시스템 메시지를 이용합니다.

역할 지시 프롬프트의 효과는 다양합니다. 모델이 **주어진 역할에 맞는 어조와 표현**을 사용하게 되므로 응답이 더 몰입감 있고 일관되게 나옵니다. 또한 **설명이나 추론이 필요한 과제**에서 전문가 역할을 부여하면, 일반적인 답변보다 **더 체계적이고 정확한 답변**을 얻을 수 있는 경우가 많습니다. 예를 들어 "너는 수학 선생님"이라고 지시하고 문제를 물어보면, 학생에게 설명하듯 자세하게 풀어주는 경향이 있습니다. 역할 프롬프트는 **상황에 특화된 조언, 대화 시뮬레이션, 특정 문체로 글쓰기** 등 폭넓은 응용이 가능하기 때문에 실무에서도 자주 활용됩니다.

Note: 역할을 부여할 때는 지나치게 개인적이거나 민감한 역할(예: 특정 정치인, 실제 개인 등)은 피하고, 전문 직업인, 조력자, 상담가 등의 중립적이고 모델이 잘 학습했을 법한 역할을 선택하는 것이 좋습니다. 역할 지시 프롬프트는 편향된 답변을 강화할 위험도 있으므로, 필요한 경우 추가 지침(예: "객관적으로 답변해줘")을 함께 주어 균형 잡힌 응답을 얻도록 합니다.

4.2. 프롬프트 예시

다양한 역할 지시 프롬프트의 **예시**를 살펴보겠습니다. 아래에는 세 가지 상황(튜터, 상담사, 면접관)에 대해 모델에게 역할을 부여한 프롬프트와 그 기대 응답의 특징을 정리했습니다:

##실습 3.4. OpenAI Playground 시스템 메시지와 사용자 프롬프트 실습

- 튜터 역할 예시:

시스템 메세지: "당신은 수학 과외 선생님입니다."

사용자 프롬프트: "피타고라스의 정리에 대해 초보자도 이해할 수 있게 설명해 주세요."

예상 응답: 모델은 **친절하고 쉬운 어조**로, 피타고라스 정리의 개념을 비유와 함께 풀어서 설명할 것입니다. 예를 들면, "피타고라스 정리는 직각삼각형에서 가장 긴 변(빗변)의 길이를 찾는 공식이에요..."처럼 **교사가 학생에게 설명하듯** 답변합니다. "부산 출신의 20대 남자로 사투리를 사용합니다." 등 역할을 추가할 수 있습니다.

- **상담사 역할 예시:**

시스템 메세지: "당신은 5년 차 직장인들을 돕는 커리어 상담사입니다. 최근 번아웃을 느끼는 내담자에게 조언을 대화체로 제공하세요."

사용자 프롬프트: "저는 고등학교 선생님인데요. 요즘 교실에 들어가는 게 힘들어졌어요."

예상 응답: 모델은 **공감적이고 조언자적인 말투**로 답할 것입니다. 예를 들어 "많이 지치셨군요. 그 동안 열심히 해오셨다는 증거예요..."와 같이 상대방의 감정을 인정하고 조언하는 **상담 대화 형식**의 답변을 생성합니다. 톤은 부드럽고 위로하는 느낌이 될 것입니다.

- **면접관 역할 예시:**

시스템 메세지: "당신은 IT 스타트업의 면접관입니다. 신입 소프트웨어 엔지니어 지원자에게 하나의 기술 면접 질문을 해보세요."

예상 응답: 모델은 **공손하면서도 전문적인 어조**로 면접 질문을 한 가지 제시할 것입니다. 예를 들어 "안녕하세요. 저희와 함께 일하게 된다면 객체지향 프로그래밍(OOP)을 많이 사용하게 될 텐데, OOP의 4가지 핵심 개념을 설명해 주실 수 있을까요?" 같은 질문이 나올 수 있습니다. 역할 지시에 따라 실제 면접관처럼 인사하고 난이도 있는 기술 질문을 합니다.

위 예시들처럼, 같은 모델이라도 어떤 역할을 부여하느냐에 따라 출력 내용과 말투가 크게 달라집니다. 이러한 역할 지시 프롬프트를 활용하면 하나의 모델을 때로는 선생님처럼, 때로는 전문 상담가나 인터뷰어처럼 활용할 수 있어 실무 활용도가 높습니다.

4.3. Playground 실습 가이드

이제 역할 지시 프롬프트를 직접 실습해보겠습니다. 다음 절차에 따라 다양한 역할을 적용한 프롬프트를 테스트해 보세요:

1. **역할 프롬프트 작성:** 앞서 제시한 튜터, 상담사, 면접관 역할 프롬프트 중 하나를 선택하여 Playground 입력 창에 입력합니다. 원하는 경우 다른 역할(예: "당신은 엄격한 편집자입니다...")을 자유롭게 정해 작성해도 좋습니다. 중요한 것은 "**당신은 ...입니다**"로 시작하여 역할과 맥락을 분명히 설명하는 것입니다.

2. **질문 추가:** 역할을 부여한 다음, 그 **역할로써 수행할 구체적 요청이나 질문**을 이어서 써야 합니다. 예를 들어 역할 문장 다음에 "그 상황에서 X를 해주세요"처럼 실제 작업 지시를 함께 작성합니다. (예시: "당신은 의사입니다. 환자의 증상을 듣고 조언해주세요:")
3. **응답 생성:** **Submit**을 눌러 모델의 응답을 받아봅니다. 모델이 해당 역할에 어울리는 말투와 정보를 활용해서 답변하는지 관찰하세요. 튜터 역할이면 친절할 설명, 면접관 역할이면 질문 형태의 출력 등이 나오는지 확인합니다.
4. **출력 검토:** 모델의 답변 내용과 톤이 요청한 역할과 일치하는지 평가합니다. 필요하다면 프롬프트를 약간 수정해볼 수 있습니다. (예: 더 격식을 차린 말투를 원하면 "격식을 갖춰 말해줘"를 추가)
5. **다른 역할 시도:** 한 가지 역할 실험이 끝났다면, 이번에는 다른 역할 프롬프트를 입력해 보세요. 예를 들어 먼저 튜터로 실행했다면, 이어서 상담사 프롬프트를 입력하고 결과를 비교합니다. 각 역할마다 모델의 어조, 사용 단어, 답변 구조가 어떻게 달라지는지 비교하며 살펴봅니다.
6. **응용 실습:** 역할 지시와 다른 프롬프트 기술을 결합해보세요. 예를 들어 **Few-shot + 역할 지시**를 함께 쓰는 것입니다. "당신은 영문 교정 전문가입니다."로 시작하고, 그 아래에 몇 줄의 잘못된 영어 문장과 교정 예시를 few-shot으로 제공한 뒤 새로운 문장을 교정하게 하는 식입니다. 이러한 복합 프롬프트로 모델의 성능을 극대화하는 방법도 시도해봅니다.

노트: Playground에서는 한 번 응답을 받고 나서 대화를 이어가는 형태로 사용할 수도 있습니다. 예컨대 역할을 지정한 후 추가 질문을 이어나가면, 모델은 여전히 그 역할을 유지하려고 합니다. 이런 맥락 유지는 OpenAI Chat API의 기본 동작이지만, Playground의 단일 프롬프트 모드에서는 이전 응답을 복사해 넣지 않으면 맥락이 유지되지 않으므로, 연속 대화를 실험하고 싶다면 매번 역할 지시를 포함하여 프롬프트를 작성해야 합니다.

5. 출력 포맷 제어 실습

프롬프트 엔지니어링에서는 **모델의 답변 내용**뿐 아니라 **출력 형식**을 제어하는 것도 매우 중요합니다. 원하는 출력 형식을 명확히 지시함으로써, 결과를 후속 처리하거나 읽기 쉽게 만들 수 있습니다. 여기서는 **목록 형태, 표 형태, JSON 형태**의 출력물을 얻도록 프롬프트를 설계하고 실습합니다. (이미지, HTML 등 다른 형식도 가능하지만, 제한된 시간 내에 대표적인 예시들만 다룹니다.)

5.1. 목록(List) 형식 출력

5.1.1. 개념 설명

모델에게 **목록(list)** 형태로 답변을 출력하도록 요구할 수 있습니다. 예를 들어 "항목을 나열해 달라"고 하면 모델은 보통 **번호나 불릿(bullet)**을 사용하여 여러 줄의 목록을 생성합니다. 이러한 목록 출력은 요점을 정리하거나 절차를 단계별로 제시할 때 유용합니다.

모델은 Markdown 문법의 목록 표기도 이해하기 때문에, "1., 2., 3."과 같은 **번호 매긴 목록(ordered list)**이나 "-", "." 기호를 사용하는 **불릿 목록(unordered list)** 모두 만들어낼 수 있습니다. 사용자가 특별히 지정하지 않으면 일반적으로 불릿 목록을 사용하기도 합니다. **프롬프트에서 직접 형식을 지정**할 수도 있는데, 예를 들어 "각 답변을 번호로 출력해줘"라고 하면 번호 목록을 만들고, "- 기호를 붙여 목록으로 써줘"라고 하면 불릿 목록을 만들 것입니다.

5.1.2. 프롬프트 예시

다음은 모델에게 **목록 형식의 답변**을 요구하는 프롬프트 예시입니다. 실무 상황으로 "효율적인 시간 관리 팁"을 묻되, **다중 항목**을 요구하여 목록으로 제시되도록 합니다:

##실습 3.5. OpenAI Playground 실습

예시 프롬프트: 효율적인 시간 관리 팁 5가지를 목록 형식으로 알려주세요.

프롬프트에서 "5가지" 그리고 "목록 형식"이라는 단어를 사용해, **답변에 5개의 항목이 있는 목록**을 기대하고 있음을 모델에게 전달합니다. 모델은 이에 따라 각 항목을 개별 줄로 구분하여 답할 것입니다.

예시 응답 (불릿 또는 번호 목록으로 출력):

1. 목표와 우선순위를 명확히 정합니다.
2. 일정을 미리 계획하고 캘린더를 활용합니다.
3. 집중을 방해하는 요소(전화, 알림)를 차단합니다.
4. 유사한 업무는 묶어서 한 번에 처리합니다.
5. 규칙적인 휴식으로 생산성을 유지합니다.

위 응답은 번호가 매겨진 목록 형태로, 모델이 다섯 가지 팁을 나열한 것입니다. 각 항목은 한 문장으로 간략히 작성되어 읽기 쉽고, 요구한 형식(5가지 목록)에 부합합니다.

(Playground 상에서는 위와 같이 텍스트로 보이며, ChatGPT UI처럼 특수한 렌더링은 없지만, 내용상 목록임을 쉽게 알 수 있습니다.)

예시 프롬프트: 효율적인 시간 관리 팁 5가지를 불릿 목록으로 알려주세요.

5.1.3. Playground 실습 가이드

목록 형식 출력을 얻기 위한 프롬프트를 직접 입력해 보고 결과를 확인해봅시다:

1. **프롬프트 입력:** Playground에 **예시 프롬프트**와 같이 원하는 주제와 항목 수를 명시하여 입력합니다. 예를 들어 " **효율적인 공부 방법 3가지를 불릿 목록으로 알려줘.** "라고 입력해보세요. ("**불릿 목록**"이나 "**번호 목록**" 등의 **표현으로 형식을 분명히 지시**할 수 있습니다.)
2. **응답 생성:** **Submit**을 눌러 모델의 응답을 봅니다. 모델이 여러 항목을 각기 다른 줄로 나열했는지 확인합니다. 정상이라면 1, 2, 3 또는 기호로 시작하는 여러 줄의 답변을 얻게 됩니다.
3. **형식 검토:** 모델이 출력한 목록이 지시한 형식과 일치하는지 살펴보세요. 혹시 한 줄에 다 몰아서 작성했거나 원하는 개수와 다르다면, 프롬프트를 더 구체적으로 수정해볼 수 있습니다. (예: "한 줄에 하나씩 번호를 매겨..." 같이 지시 추가)
4. **다른 지시 실험:** 이번에는 동일한 질문을 하되 "**번호 대신 불릿 기호**로 목록을 만들어줘"라고 프롬프트를 변경해보세요. 모델이 나 기호로 된 목록을 출력하는지 확인합니다. 또한 항목 수를 늘리거나 줄여보면서 요구한 개수만큼 응답하는지도 테스트해보세요.
5. **응용 및 활용:** 얻은 목록 결과를 복사하여 노트나 보고서 등에 활용할 수 있습니다. Playground에서 얻은 텍스트는 Markdown 형식 문법이 포함된 그대로 출력되므로, 마크다운을 지원하는 에디터나 뷰어에 붙여넣으면 보기 좋게 렌더링할 수 있습니다 (불릿 기호 등이 제대로 표시됨).

추가 팁: 목록형 답변이 잘 나오지 않을 때는 프롬프트에 직접 예시를 포함하는 것도 방법입니다.

5.2. 표(Table) 형식 출력

5.2.1. 개념 설명

모델은 간단한 표 형태의 출력도 생성할 수 있습니다. **표 형식 출력**은 데이터를 행과 열로 정리하거나 비교 내용을 구조화할 때 유용합니다. 프롬프트에서 "**표로 보여줘**", "**테이블 형태로 정리해줘**"와 같이 요청하면, 모델은 Markdown 표 형태의 텍스트나 CSV 유사 형식으로 답변을 구성하려고 시도합니다.

Markdown 표 형식은 파이프(**|**) 문자로 컬럼을 구분하고, 하이픈(**-**)으로 헤더 구분선을 넣는 방식입니다. 예를 들어 "이름 | 나이 | 직업"과 같은 헤더 행과 구분선, 데이터 행들을 모

델이 만들어낼 수 있습니다. ChatGPT 인터페이스에서는 이러한 Markdown 표를 실제 표로 예쁘게 보여주지만, Playground에서는 결과가 **텍스트 형태**로 나타납니다. 그래도 파이프로 구분된 형식이므로 복사해서 마크다운 뷰어나 Excel에 변환하면 표로 활용할 수 있습니다.

프롬프트 설계 시 **열(Column) 제목**이나 **행(Row) 개수**를 지정하면 보다 정확한 표를 얻을 수 있습니다. 모델은 요청한 대로 표를 생성하나, 내용은 허구로 채워질 수 있다는 점을 기억하세요 (특히 예시 데이터를 주지 않으면 임의의 값을 생성함).

5.2.2. 프롬프트 예시

아래는 모델에게 **표 형태 출력**을 요청하는 프롬프트 예시입니다. 재고 관리 시나리오를 가정하여, 세 가지 제품의 정보를 표로 정리해 달라고 해보겠습니다:

##실습 3.6. OpenAI Playground 실습

예시 프롬프트: 세 가지 전자제품의 제품명, 가격(원), 재고량(개) 정보를 표로 만들어 보여주세요.

모델은 "제품명, 가격, 재고량"을 컬럼 헤더로, 세 가지 제품에 대한 정보를 행으로 채운 표를 만들어낼 것입니다.

예시 응답:

제품명	가격(원)	재고량(개)
제품A	10,000	50
제품B	20,000	20
제품C	15,000	0

위 응답은 Markdown 형식으로 구성된 표입니다. 첫 행은 헤더, 둘째 행의 `----`은 헤더와 데이터를 구분하는 선, 이후 각 행에 제품명, 가격, 재고량이 파이프(`|`)로 구분되어 나열되어 있습니다. Playground에서는 이 그대로 텍스트로 표시되지만, 시각적으로 각 열이 구분되므로 표 구조를 이해할 수 있습니다. 실제 데이터가 없으므로 가격과 재고 수치는 모델이 임의로 만든 값입니다. (실무에서는 모델이 만든 데이터를 그대로 쓰기보다는, 형식만 활용하고 실제 데이터는 나중에 채워넣는 식으로 활용 가능성을 유념하세요.)

5.2.3. Playground 실습 가이드

이제 표 형식 출력을 Playground에서 얻어보는 실습을 진행합니다:

1. **프롬프트 입력:** 위 예시 **프롬프트**와 같이, 표로 만들고 싶은 정보의 열과 내용을 설명하는 문장을 입력합니다. 다른 예로 "프로젝트 일정(단계, 책임자, 마감일)을 표로 작성해줘."처럼 열 이름(단계, 책임자, 마감일)을 명시하고 표 출력을 요구할 수 있습니다.
2. **응답 확인:** **Submit**을 눌러 모델의 출력을 확인합니다. 모델이 Markdown 표 형태의 텍스트를 생성했다면, 파이프(`|`) 문자를 사용한 행들이 나타날 것입니다. 헤더 행과 구분선, 데이터 행들이 제대로 구성되었는지 확인하세요.
3. **형식 검사:** 표의 형태가 어긋나거나 열 개수가 맞지 않는다면, 프롬프트를 더 구체화해야 합니다. 예를 들어 열 이름을 한 문장에 콤마로 구분해서 나열하거나, "각 열은 '단계', '책임자', '마감일' 입니다"처럼 명확히 지정해주는 방법이 있습니다.
4. **데이터 검토:** 모델이 채운 데이터는 사실 임의의 예시입니다. 중요한 것은 **형식**입니다. 결과 표를 복사하여 Markdown을 지원하는 곳(예: GitHub README 또는 Colab 마크다운 셀)에 붙여넣으면 실제 표처럼 볼 수 있습니다. 또는 파이프 구분을 활용해 CSV로 바꾸는 것도 가능합니다.
5. **응용 실습:** 프롬프트에 나열된 항목 수를 바꿔보거나, 열을 추가/삭제하면서 모델이 동적으로 표를 생성하는지 실험해보세요. 또한 "**CSV 형식으로 출력해줘**"라고 요청하면 쉼표로 구분된 값 형태로 출력할 수도 있습니다. 일부 사용자는 차라리 CSV나 JSON 형태로 출력하게 하여 데이터를 다루기 쉽게 하기도 합니다.

주의: Playground에서 표 출력 결과를 바로 Excel 등에 복사/붙여넣기 하면 한 칸에 다 들어가 버릴 수 있습니다. 이런 경우, 텍스트를 복사한 후 Excel의 텍스트 나누기 기능을 사용하여 `|`를 구분자로 지정하면 열별로 나눌 수 있습니다. 혹은 앞서 언급했듯 한 번 Markdown에 렌더링한 후 표 자체를 복사하는 방법도 있습니다.

5.3. JSON 형식 출력

5.3.1. 개념 설명

JSON(JavaScript Object Notation)은 데이터 교환을 위한 **구조화된 경량 형식**으로, 키-값 쌍으로 이루어진 계층적 데이터를 표현하는 데 널리 사용됩니다. **JSON 형식 출력**을 모델에게 요청하면, 모델은 주어진 정보를 JSON 객체나 배열 형태로 생성합니다. 이 형식은 프로그래밍 언어나 스크립트에서 바로 파싱하여 사용할 수 있기 때문에, 모델 응답을 다른 시스템과 연계하거나 후처리할 때 유용합니다.

프롬프트에서 "**JSON으로 답해줘**", "**결과를 JSON 형식으로 출력**" 등의 지시를 내리면 모델은 응답 전체를 `{ ... }` 나 `[ ... ]` 와 같은 JSON 표기법으로 감싸서 출력하려 합니다. 다만

ChatGPT 계열 모델들은 답변을 전달할 때 종종 답변을 Markdown의 코드 블록으로 감싸곤 합니다. 예를 들어, 실제 내용은 JSON이지만, 출력은 아래처럼 되어 있을 수 있습니다:

```
```json
```

```
{
 "key": "value",
```

```
 ...
```

```
}
```

```
```
```

위와 같이 json으로 시작하는 코드블록 형식으로 응답 하면, 사람에게서는 JSON임을 명확히 보여주지만, 프로그래밍적으로 이 문자열을 바로 파싱하려 하면 문법 에러가 날 수 있습니다. 따라서 프로그래밍 환경(Colab 등)에서 JSON 응답을 받을 경우에는 이러한 코드블럭 표기나 불필요한 설명 텍스트를 제거하고 순수 JSON 문자열만 추출하는 전처리가 필요합니다.

물론, 프롬프트에 "코드 블럭 없이 출력해줘"라고 언급하면 모델이 ``` 표기를 안쓰기도 하지만 완벽하지 않으므로, 안전하게는 클라이언트 측에서 문자열 처리를 하는 것이 권장됩니다. 아래 Colab 실습에서 이러한 처리를 다룰 것입니다.

5.3.2. 프롬프트 예시

JSON 출력을 요구하는 프롬프트도 실습해보겠습니다. 예를 들어 두 도시의 날씨 정보를 JSON으로 달라고 요청해 보겠습니다 (모델이 임의의 간단한 날씨 데이터를 만들도록):

##실습 3.7. OpenAI Playground 실습

예시 프롬프트: 서울과 부산 두 도시의 현재 날씨(온도, 날씨 상태)를 JSON 배열로 알려주세요.

모델은 두 도시의 날씨를 담은 JSON 데이터를 만들어낼 것입니다. 예시 응답은 대략 다음과 같을 수 있습니다:

```
[  
  {  
    "city": "서울",  
    "temperature": 18,  
    "weather": "맑음"  
  },  
  {  
    "city": "부산",
```

```
"temperature": 20,  
"weather": "흐림"  
}  
]
```

위와 같이 모델이 출력하면 내용 자체는 올바른 JSON이지만, 실제 Playground에서는 **json 코드블럭으로 감싸서** 나타냅니다. 그럼 출력 창에는 json으로 시작하고 ``로 끝나는 형태로 보일 것입니다. 중요한 것은 모델이 요구한 대로 **키(name)-값(value)** 구조를 맞춰서 데이터를 생성했다는 점입니다. 이 JSON 예시는 두 개의 객체(서울, 부산)를 배열 [...] 로 묶은 형식으로, 각 객체는 **city**, **temperature**, **weather** 키를 가지고 있습니다. (온도 값 등은 모델이 임의로 제공한 것입니다.)

5.3.3. Playground 실습 가이드

JSON 형식 출력을 Playground에서 실습할 때 유의사항은, **응답을 눈으로 보고 확인**하는 것과 **프로그램에서 활용하는 것**이 약간 다를 수 있다는 점입니다. 우선 Playground에서 제대로 JSON이 생성되는지 확인해보고, 이후 Colab에서 해당 출력을 파싱하는 연습을 해보겠습니다.

1. **프롬프트 입력:** 위 예시 프롬프트처럼 JSON 출력을 요구하는 문장을 입력합니다. 형식에 대한 요구사항을 분명히 하기 위해 **"JSON 배열로"** 또는 **"JSON 객체로"** 등으로 구체화하는 것이 좋습니다. 예: "학생 2명의 이름과 나이를 JSON 형식으로 알려줘."
2. **응답 확인:** **Submit**을 눌러 응답을 봅니다. 모델이 JSON 형식에 맞게 답했는지 우선 내용적으로 검토합니다. 중괄호 {}, 대괄호 [] 가 잘 열고 닫혔는지, 키 이름에 따옴표가 붙었는지, 콤마로 구분이 되었는지 등 **JSON 문법 구조를 확인**하세요. 눈으로 볼 때 이상없어 보여도, 실제로 파싱해보기 전까지는 확신할 수 없으니 다음 단계를 진행합니다. 응답에서 code도 확인해 봅니다.
3. **응답 복사:** 모델 응답을 그대로 복사(Ctrl+C)합니다. 복사할 때 ``json 등의 부분까지 모두 포함되었을 수 있으니 유의합니다.
4. **Colab 준비:** 이제 Colab에서 이 JSON 응답을 파싱하는 코드를 실행해볼 것입니다. Colab 환경을 열고 필요한 패키지를 준비합니다(다음 절차에서 상세히 안내).

Note: Playground에서 JSON 응답을 얻을 때, 간혹 모델이 추가 설명을 덧붙이거나 JSON 직전에 한두 문장 자연어로 말하는 경우가 있습니다. 예: "다음은 JSON 결과입니다:" 같은 문장이 앞에 나오고 그 다음 줄부터 ``json 코드블럭이 시작될 수 있습니다. 이러한 경우 프롬프트를 더 엄격히 하거나, 파싱 전에 해당 문구를 제거해야 합니다. 가

능하면 "**오로지 JSON만 답변해줘**" 라고 프롬프트 끝에 강조하면 그런 경우를 줄일 수 있습니다. 그래서 시스템 메시지의 **Response Format**을 **json_object**로 하는 것이 응답의 품질을 높일 수 있습니다.

6. Colab 실습 가이드 및 Python 코드 예시

마지막으로, Playground에서 얻은 (혹은 API를 통해 직접 받은) JSON 형식 응답을 **Google Colab**에서 파싱하여 사용하는 방법을 연습합니다. 이 과정에서는 OpenAI의 Python 클라이언트를 통해 직접 모델 호출을 하고, 응답을 처리해보겠습니다.

##실습 3.8. Google Colab 실습

Colab에서 아래 단계를 따라주세요:

1. **OpenAI 라이브러리 설치/업데이트:** Colab 노트북 코드 셀에 아래 명령을 입력해 OpenAI Python SDK를 최신 버전(≥ 1.6)으로 설치합니다. (OpenAI 1.6 이상 버전에서는 응답 객체 구조와 함수 호출 등이 변경되었으므로 최신 버전 사용을 권장합니다.)

```
!pip install -U openai
```

2. **API Key 설정:** OpenAI API를 사용하려면 API 키가 필요합니다. Colab에서 키를 노출하지 않고 사용하기 위해 **Secrets**에 미리 저장해두었다면, 다음 코드로 불러올 수 있습니다:

```
from google.colab import userdata
import os

# Colab에 저장한 Secret에서 API 키 가져와 Linux 가상 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')
```

위 코드로 `openai.api_key` 가 설정되면, 이후 OpenAI API 호출 시 해당 키가 사용됩니다.

3. **OpenAI 클라이언트 초기화:** OpenAI Python SDK v1.6부터는 `openai.ChatCompletion.create` 대신 통합 클라이언트 객체를 사용할 수 있습니다. 다음과 같이 클라이언트를 생성합니다:

```
import openai
client = openai.OpenAI()
```

(이때 `openai.api_key` 가 설정되어 있으므로 `client` 객체는 인증된 상태로 만들어집니다.)

4. **프롬프트 준비:** 이제 Colab에서 모델에게 보낼 동일한 프롬프트를 준비합니다. 예시로, 아까 Playground에서 시도한 프롬프트를 그대로 변수에 넣습니다:

```
prompt = "서울과 부산 두 도시의 현재 날씨(온도, 날씨 상태)를 JSON 배열로 알려주세요."
```

5. **API 호출 및 응답 받기:** 클라이언트의 `chat_complete` (또는 유사 함수)를 사용해 GPT-4o-mini 모델에 프롬프트를 보냅니다:

```
response = client.chat.completions.create(
    messages=[
        {"role": "user", "content": prompt}
    ],
    model="gpt-4o-mini",
    response_format={
        "type": "json_object"
    })
```

이 호출이 완료되면 `response` 객체에 모델의 응답이 담깁니다.

6. **콘텐츠 추출:** 응답 객체에서 실제 텍스트 응답(content)을 추출합니다. OpenAI SDK에서는 아래와 같이 할 수 있습니다:

```
content = response.choices[0].message.content
print(content) # 우선 출력 내용을 확인해봅니다.
```

여기서 `content` 변수에는 모델의 응답 문자열이 저장됩니다. 앞서 논의한 대로, 이 문자열에는 `json`이 포함되어 있을 수 있습니다. `'print'`로 출력해보고 결과를 살펴보세요. (`json`으로 시작하는 형식일 것입니다.)

7. **JSON 파싱:** 파이썬의 내장 `json` 모듈을 이용해, 문자열을 실제 데이터 구조로 변환합니다:

```
import json
data = json.loads(content)
```

8. **파싱 결과 확인:** 파싱된 `data` 를 출력하거나 타입을 확인하여 제대로 파싱되었는지 검증합니다:

```
print(data)
print(type(data))
```

9. **추가 활용:** 이제 `data` 객체를 가지고 파이썬 코드로 자유롭게 조작할 수 있습니다. 예컨대,

```
for city in data.keys():
    print(city, "의 날씨는", data[city]["날씨상태"], "이고 온도는", data[city]
["온도"], "입니다.")
```

처럼 각 도시의 날씨를 문장으로 출력하는 후처리도 할 수 있습니다.

위의 과정을 종합하면 아래와 같은 Python 코드 예시가 됩니다:

```
!pip install -U openai
```

```
from google.colab import userdata
import os
```

```
# 1. Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')
```

```
# 2. 클라이언트 초기화
client = openai.OpenAI()
```

```
# 3. 프롬프트 정의
prompt = "서울과 부산 두 도시의 현재 날씨(온도, 날씨 상태)를 JSON 배열로 알려주세요."
```

```
# 4. GPT-4o-mini 모델 호출하여 json_object로 응답 얻기
response = client.chat.completions.create(
    messages=[
        {"role": "user", "content": prompt}
    ],
    model="gpt-4o-mini",
    response_format={
        "type": "json_object"
    })
```

```
# 5. 응답에서 콘텐츠 추출
content = response.choices[0].message.content
print(content) # 우선 출력 내용을 확인해봅니다.

# 6. JSON 파싱
import json
data = json.loads(content)

# 7. 파싱된 데이터 활용 예시 출력
for city in data.keys():
    print(city, "의 날씨는", data[city]["날씨상태"], "이고 온도는", data[city]["온도"], "입니다.")
```

7. 정리

이상으로 **프롬프트 엔지니어링 기초 개념 및 구조적 설계 실습**을 모두 다뤘습니다. 이번 세션에서 배운 내용을 요약하면 다음과 같습니다:

- **Zero-shot 프롬프트:** 예시 없이 직접 질문하거나 지시하는 방식. 간결하지만 명료하게 작성해야 하며, 간단한 작업에 유용합니다.
- **Few-shot 프롬프트:** 두 개 이상의 예시를 프롬프트에 포함하여 모델이 패턴을 학습하도록 하는 방식이고 복잡한 작업이나 포맷 일관성이 중요할 때 효과적이며, 모델의 출력 정확도와 일관성을 높여줍니다.
- **역할 지시 프롬프트:** 모델에게 특정 역할을 부여하여 해당 페르소나에 맞는 어조와 정보를 활용하게 하는 기법입니다. 튜터, 상담사, 면접관 등 다양한 역할로 응답을 컨트롤하여 더 현실감 있고 전문적인 답변을 얻을 수 있습니다.
- **출력 포맷 제어:** 모델의 답변 형식을 목록, 표, JSON 등 원하는 형태로 지정하는 방법. 목록은 "-"이나 번호로 항목화된 답을 얻고 싶을 때, **표**는 행열 구조로 정리할 때, **JSON**은 구조화된 데이터가 필요할 때 활용합니다. 프롬프트에 형식을 명시적으로 요구하면 모델이 그에 맞춰 답변하며, 특히 JSON은 후속 프로그래밍 처리를 용이하게 해줍니다. JSON 응답을 코드에서 다룰 때는 **마크다운 코드블럭 제거 등 전처리**가 필요합니다.
- **모델 선택과 프롬프트 튜닝:** 경량 모델(GPT-4o-mini)은 속도가 빠르나 복잡한 요구사항에 오류가 생길 수 있으므로, 필요 시 더 강력한 모델(GPT-4o)을 활용하거나 프롬프트를 세밀하게 조정하는 것이 중요합니다.